Defiende el servidor: versionar y recuperar un clasificador normal/ataque

MLOps 2026-II · Universidad Externado de Colombia · David Santiago Briñez Forero, Mattias Morales Serna · Grupo B · Código: github.com/davidbrinez30/defiende-el-servidor-unsw-nb15 · Modelos: huggingface.co/d4v1dsbf/defiende-el-servidor-unsw-nb15

Resumen. Se construyó un clasificador de conexiones de red (0 normal, 1 ataque) con UNSW-NB15 y se versionaron código (Git), datos (DVC), experimentos (MLflow) y modelos (Hugging Face Hub) de modo que cada versión se recupera por identificadores y hashes. v1 es una regresión logística; v2, un boosting de árboles elegido con validación. Con la misma regla de umbral, v2 detecta 95,42 % de los ataques frente a 91,21 % de v1 en validación. En el test publicado, v2 mantiene 96,93 % de recall pero la tasa de falsos positivos sube a **18,33 %** (meta de validación: 5 %); se conserva y se explica. La prueba de recuperación en un entorno nuevo resultó **aprobada** para ambas versiones.

1. Datos y limpieza

Se usaron las particiones publicadas: entrenamiento (175.341 filas, 45 columnas) y test (82.332), copia fijada de un mirror de terceros (commit `44bbd02b`, descargada el 2026-10-11); los SHA-256 coinciden con los del enunciado y se verifican en cada carga (no autentican contra UNSW). No hay valores faltantes; «-» en `service` es una categoría con significado. Entradas del modelo: 42 variables (39 numéricas, 3 categóricas); se excluyen `label` (respuesta), `attack_cat` (revela la respuesta) e `id` (separa las clases con AUC 0,95 solo por cómo viene ordenado el archivo).

- **Duplicados por entradas:** 42,4 % de las filas de entrenamiento repiten las entradas de otra (101.040 patrones distintos); en test, 34,5 %.
- **Contradicciones:** 229 grupos de entradas iguales con etiquetas distintas (940 filas, 0,54 %). Política: no se eliminan, imputan ni recortan; se mantienen juntos en una misma partición y se reconocen como error irreducible.
- **Particiones sin fuga:** `StratifiedGroupKFold` (5 pliegues, semilla 42, grupo = huella de las 42 entradas); un pliegue es validación: 140.274 filas de entrenamiento y 35.067 de validación, 0 patrones compartidos. Codificación, escalado y log(1+x) viven dentro del `Pipeline` y se ajustan solo con entrenamiento.
- **Solapamiento con el test (auditado sin etiquetas):** 10,4 % de las filas del test (8.541) tienen entradas idénticas a alguna de entrenamiento o validación; el 89,6 % restante son entradas nuevas.

DVC conserva dos versiones de datos: **originales** (`data/raw/*.dvc`, md5 `e55caaba…` y `e0beea40…`) y **preparadas** (`train.parquet`, `val.parquet`, `split_assignment.csv`, en `dvc.lock`), ambas recuperables desde un remote HTTP público.

2. Experimentos

Siete configuraciones de cuatro familias (baseline, regresión logística, Random Forest, boosting) sobre las **mismas particiones**; cada una cambia una sola decisión respecto a su referencia, con hipótesis y criterio medible escritos antes de entrenar (commit `68d3332`). MLflow registra hipótesis, parámetros, semilla, commit, versión de datos, métricas y el modelo completo (un run por experimento, exportados a `evidencias/mlflow/`). Métricas con ataque como clase positiva, en el umbral de cada experimento (mayor recall con FPR ≤ 5 % en validación).

ID	Cambio	Familia	Recall	Precision	Macro-F1	ROC-AUC	¿Hipótesis?
E0	Referencia sencilla: predecir siempre «ataque» (no aprende nada).	baseline	1,0000	0,6806	0,4050	0,5000	sí
E1	Primer modelo real: regresión logística sobre variables escaladas, sin transformar las colas.	regresion logistica	0,8977	0,9745	0,9055	0,9843	sí
E2	Preprocesamiento: aplicar log(1+x) antes de escalar.	regresion logistica	0,9121	0,9749	0,9157	0,9866	sí
E3	Equilibrar clases con class_weight='balanced'.	regresion logistica	0,9126	0,9749	0,9161	0,9868	sí
E4	Tipo de modelo: Random Forest en lugar de regresión logística (mismo preprocesamiento que E2).	random forest	0,9514	0,9759	0,9444	0,9933	sí
E5	Aumentar complejidad: boosting de árboles secuenciales en lugar de bosque aleatorio.	hist gradient boosting	0,9542	0,9760	0,9464	0,9936	sí
E6	Usar solo 25 % de las filas de entrenamiento (muestra reproducible, semilla 42).	random forest	0,9399	0,9756	0,9359	0,9918	sí

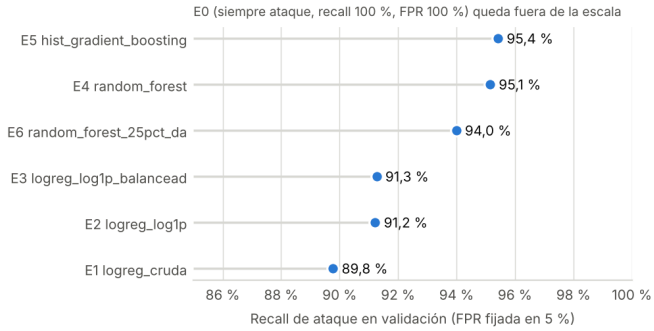


Fig. 1. Recall en validación con la FPR fijada en 5 % (todas las ejecuciones comparten partición).

Lectura. El preprocesamiento (log(1+x), E2) y los árboles (E4, E5) mejoran de forma clara y con costos distintos; el *class_weight* (E3) cambia el punto de operación a umbral 0,5 pero no el orden de las puntuaciones; con 25 % de los datos (E6) se pierde poco (0,0085 de macro-F1) y el entrenamiento es unas 4,5 veces más rápido. 7 de 7 hipótesis se cumplieron según su criterio (`experimentos.csv`) — todas —; debe leerse con cautela: se escribieron conociendo una iteración previa del proyecto (§6).

3. Versiones v1 y v2 y elección del umbral

Umbral. Regla declarada antes de ver resultados: *entre los umbrales con FPR ≤ 5 % en validación, el de mayor recall*; se predice ataque si `score_attack ≥ umbral`. Se prefirió a 0,5 porque 0,5 deja una FPR de 8,55 % en v2 (13,77 % en v1) y una falsa alarma tiene costo operativo. Denominadores cero: precision = 0 sin alertas, recall = 0 sin ataques reales, FPR = 0 sin normales reales (no ocurre en validación ni test).

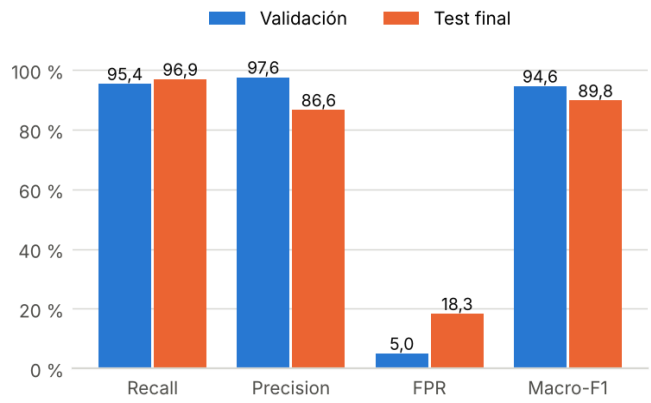
v1 (fija de antemano): regresión logística con log(1+x) (E2). **v2** (elegida solo con validación, antes de tocar el test): el experimento de modelo real con mayor macro-F1, `E5_hist_gradient_boosting`; cambio único = tipo de modelo. Frente al segundo (Random Forest) su ventaja es pequeña: 0,0021 de macro-F1 (IC 95 % 0,0004 a 0,0037, bootstrap de 500).

Validación (35.067 filas)	v1	v2	Cambio
Recall	91,21 %	95,42 %	+4,20 pts
Precision	97,49 %	97,60 %	+0,11 pts
FPR	5,00 %	5,00 %	igual (regla)
Macro-F1	0,9157	0,9464	+0,0307
Umbral	0,7774	0,6544	
Falsos negativos / falsos positivos	2.097 / 560	1.094 / 560	

Qué mejora: se detectan más ataques con las mismas falsas alarmas. **Qué cuesta:** el paquete pasa de 324 KB a 2.852 KB, el entrenamiento de 6,7 s a 14,5 s y se pierde la lectura de coeficientes. **Qué se acepta:** 1.094 ataques no detectados y 560 falsas alarmas en validación.

4. Test final

La selección se congeló (`evidencias/seleccion.json`, tag `congelado`) después de publicar v1/v2 y antes de abrir el test; la solución congelada se evaluó **una vez** sobre las 82.332 filas del archivo (ninguna excluida).



Test final	Valor	IC 95 %
Recall	96,93 %	96,76 %–97,10 %
Precision	86,63 %	86,38 %–86,92 %
FPR	18,33 %	17,96 %–18,69 %
Macro-F1	0,8979	0,8961–0,9000
VP / FP / FN / VN	43.942 / 6.781 / 1.390 / 30.219	

Fig. 2. v2 con el umbral fijado en validación.

El resultado empeora y se conserva. La FPR pasa de 5 % a 18,33 %: el umbral de validación no controla las falsas alarmas en el test. No se debe solo a entradas desconocidas: la FPR fue 17,88 % en las entradas nuevas y 29,21 % en las ya vistas. La validación (partición aleatoria agrupada del mismo archivo) no reproduce el cambio entre las particiones publicadas. Por familia de ataque, el menor recall es *Fuzzers* (79,89 %); el resto supera 94 %. Las conclusiones se limitan a estas particiones de un benchmark histórico.

5. Trazabilidad y recuperación

Cadena: **commit de Git → referencia DVC → run ID de MLflow → revisión de Hugging Face**. v1: tag `v1`, run `9c11a62d…`, HF `3c9aabf11b…`, modelo `1b7fc9ed8e…`. v2: tag `v2`, run `924822b5…`, HF `3639b0994c…`, modelo `3e4ade35db…` (identificadores completos en `entrega.md`). El mismo archivo `modelo.skops` se describe, se descarga y produce las predicciones. **Prueba de recuperación** en carpeta, entorno virtual y cachés nuevos (Windows-11-10.0.26200-SP0, Python 3.13.15): clonar, instalar dependencias fijadas (208 s), `dvc pull` desde el almacenamiento entregado (9 s) con huellas SHA-256 coincidentes, descarga de v1 y v2 de Hugging Face por SHA completo y predicción sobre 50 filas fijas de validación: y_pred idénticas y diferencia máxima de puntuación 3.9e-16 (tolerancia 1e-09) en ambas versiones; 2 de 50 filas difieren entre v1 y v2.

6. Limitaciones

- Benchmark de 2015 en laboratorio; copia de un mirror sin autenticar contra UNSW. No demuestra eficacia sobre una red actual.
- Una sola partición de validación; la ventaja de v2 sobre Random Forest es pequeña. Los IC de test tratan filas como independientes y subestiman la incertidumbre (hay entradas repetidas).
- El test difiere de la validación y la FPR se multiplica por 3,7: antes de usarlo habría que fijar el umbral con datos del dominio de despliegue o controlar la FPR con un conjunto de calibración independiente.
- Historia: el repositorio conserva una primera iteración con un pipeline distinto que ya había evaluado el test con una configuración parecida; se reconstruyó todo y ningún modelo ni umbral se eligió con el test, pero las hipótesis se escribieron conociendo esos resultados previos.
- Los paquetes skops no son reproducibles byte a byte al reconstruirse (identificadores de objeto, fechas): el hash se fija al empaquetar y las versiones se recuperan por DVC/Hugging Face.
- Uso de IA: el asistente Claude escribió y depuró código y documentación bajo dirección del estudiante; las cifras salen de scripts y la recuperación se verificó de forma automática (ver `entrega.md`).

Referencias

- Moustafa, N. y Slay, J. (2015). UNSW-NB15: a comprehensive data set for network intrusion detection systems. *MilCIS*. https://research.unsw.edu.au/projects/unsw-nb15-dataset
- Moustafa, N. y Slay, J. (2016). The evaluation of network anomaly detection systems. *Information Security Journal*, 25(1–3).
- Pedregosa et al. (2011). Scikit-learn: machine learning in Python. *JMLR* 12. · Guía de errores comunes: https://scikit-learn.org/stable/common_pitfalls.html
- DVC: https://dvc.org/doc · MLflow Tracking: https://mlflow.org/docs/latest/tracking.html · skops: https://skops.readthedocs.io
- Hugging Face Hub, descarga por revisión: https://huggingface.co/docs/huggingface_hub/guides/download · Model cards: https://huggingface.co/docs/hub/model-cards